Lecture 6: Boosting

Max G'Sell
Machine Learning II: 46-927

February 25, 2019

Announcements

- Homework 5 is due today. Homework 5 comes out tomorrow (or Wednesday...) night and is due on March 4.
- The March 4 lecture will be from Pittsburgh
- **Final exam:** The exam is Friday, March 8 from 4pm-7pm.
  - Largely conceptual questions. Many short questions.
  - Allowed 1 page (2 sides) of handwritten notes, which you must handwrite yourself.

Today

Last time we introduced tree-based models, and talked about decision trees and random forests. We introduced random forests as an approach to improving the predictive power of trees while maintaining many of their other qualities.

- Remarks about the homework
- A couple remarks about prediction in `R`
- Boosting.

Unbalanced Data

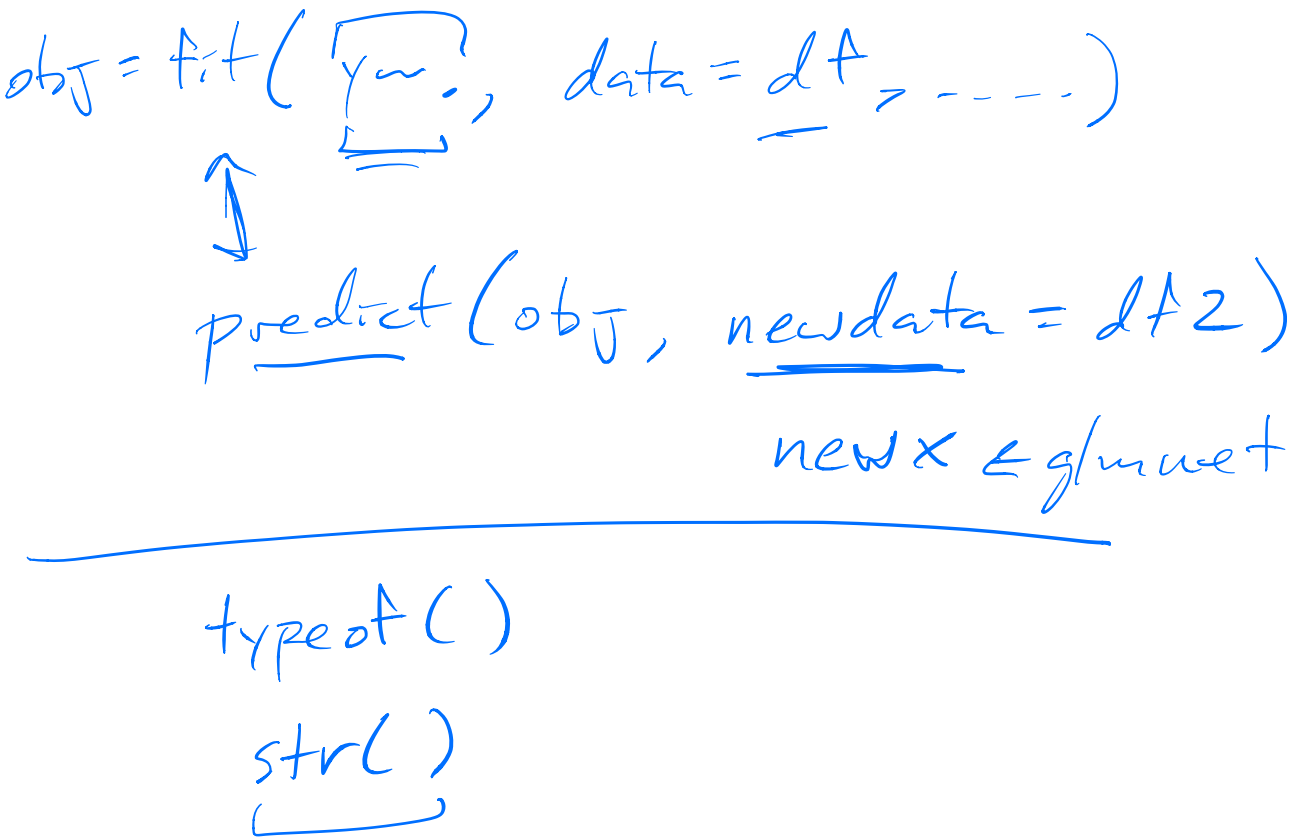
1) $loss = \sum_{i=1}^{n} w_i (y_i - \hat{f}(x_i))^2$

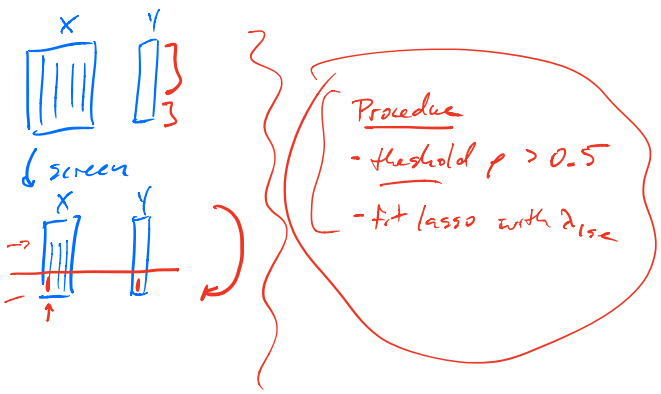
2) Upsampling &

3) Downsampling &

lam =

A couple R remarks

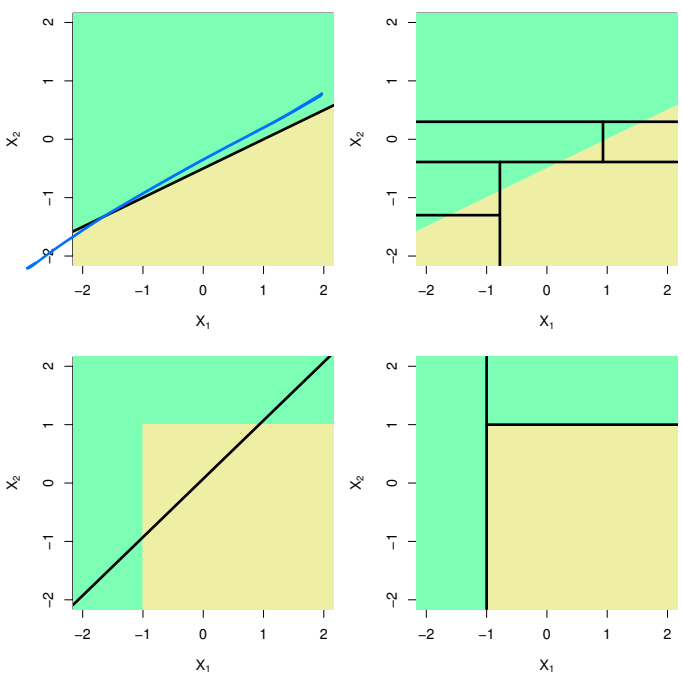



X
Y
screen
X
Y
Procedure
- threshold $\rho > 0.5$
- fit lasso with λ_{1se}

Back to last mini: Trees

You learned about regression and decision trees last lecture.

These methods approximate $\mathbb{E}(Y|X)$ or $\mathbb{P}(Y = 1|X)$ by constant values on a rectangular partition of the X space.

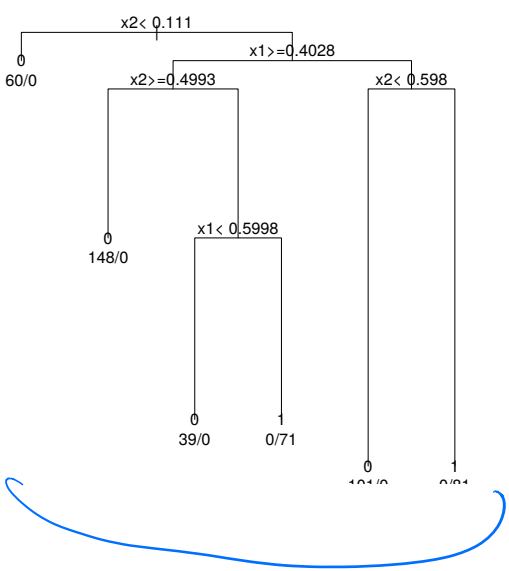


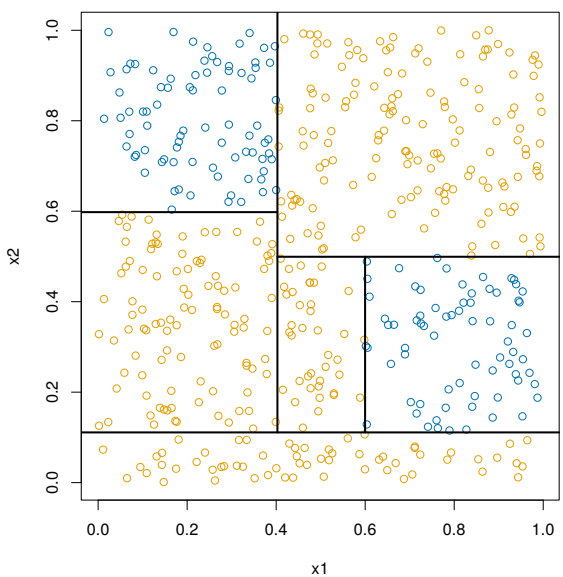
(ISL Figure 8.7)

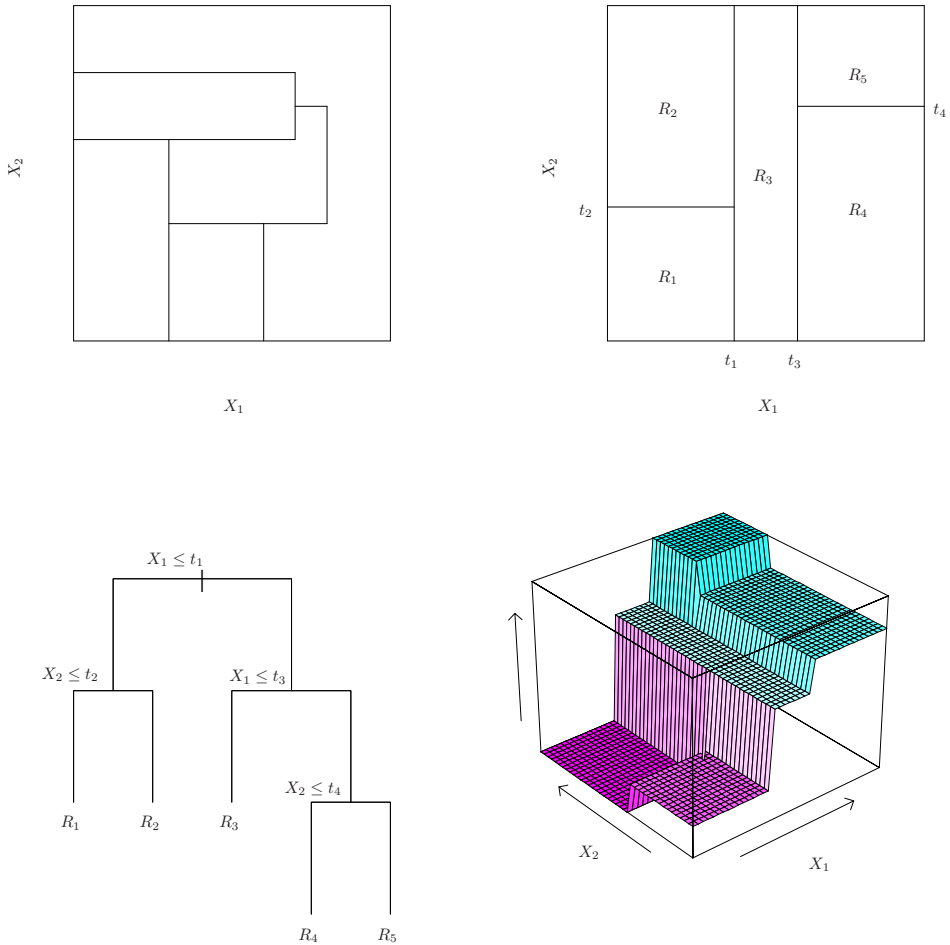
Trees

Because searching over all rectangular partitions is expensive, these methods only consider partitions that can be represented as binary trees

The binary trees are obtained through a simple greedy search







(ISL Figure 8.3)

Trees: Tuning

In terms of bias/variance:

- large/deep trees are: high var, low bias
- small/shallow trees are: low var, high bias

For decision trees, we learned to fit trees by constructing a large tree, and then pruning it back to a reasonable size.

You used cross-validation to determine the appropriate amount of tuning.

Trees

Trees are great because they are simple, interpretable, and do not require strong assumptions.

They can have very low bias, since most functions can be approximated well by a rectangular partition (eventually).

They are even **invariant to monotonic variable transformations**, and they can **capture simple interactions**!

However, they turn out to have high variance, which leads to poor prediction performance.

How can we fix this?

Bagging

We used averaging of somewhat independent trees to reduce the variance of our method.

To obtain many versions of the tree, we used Boostrapping to generate new version of the data set, and then trained a new tree on each.

This reduced variance while leaving bias essentially unaffected. Deep trees gave low bias and averaging gave low variance.

Bagging Trees

1. For $b = 1, \ldots, B$, we draw n bootstrap samples to form a new data set X^{*b}, y^{*b}.

2. For each bootstrap data set, we train a separate regression/classification tree, $\hat{f}^b$.

3. When we get a new input $x \in \mathbb{R}^p$, we form estimates $\hat{f}^b(x)$ from every tree and combine them to get an answer.

$$\hat{p}_k^{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{p}_k^b(x)$$

Then just pick the category with the highest average probability.

Random Forests

Random forests are just bagged trees, plus one more modification.

1. For $b = 1, \ldots, B$, we draw n bootstrap samples to form a new data set X^{*b}, y^{*b}.

2. For each bootstrap data set, we train a separate regression/classification tree, $\hat{f}^b$. When finding each split in the tree, only allow a random subset of variables to be considered!

3. When we get a new input $x \in \mathbb{R}^p$, we form estimates $\hat{f}^b(x)$ from every tree and combine them to get an answer.

This simple twist improves the performance of the resulting ensemble.

From Bagging to Boosting

Random forests give one way to combine a collection of *weak* predictors into a much stronger predictor.

Bagging is not specific to trees. You can use this same resampling approach in many settings.

Boosting is another method to combine many weak predictors into a stronger one.

Boosting

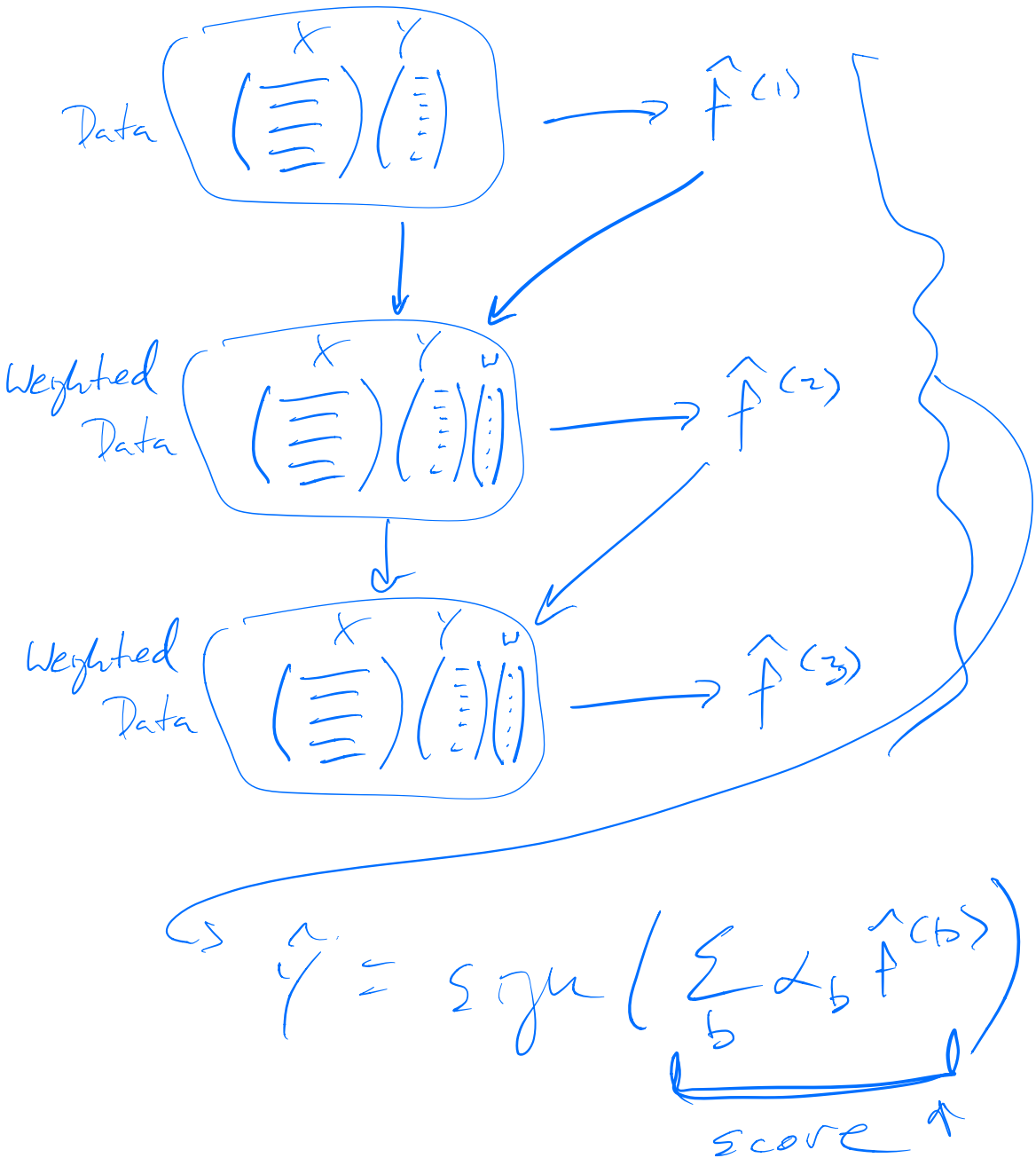
Like Bagging, Boosting is an approach to combining many weak estimators into a powerful estimator.

In *bagging*, we wanted our trees to be as independent as possible, so that the average would have low variance.

In *boosting*, we will build a sequence of simple prediction functions (usually trees). Each function will try to do well on observations that the previous functions did poorly on.

This makes the individual functions very dependent! However, we will see that it can have great performance.

Boosting Cartoon



Original Boosting Algorithm (AdaBoost)

Initialize: Weights $w_i = 1/n$

For $b = 1, \ldots, B$**:**

1. Fit classification tree $\hat{f}^b$ to the training data with weights $w_1, \ldots, w_n$.
2. Compute weighted misclassification error:

$$e_b = \frac{\sum_{i=1}^n w_i \mathbf{1}\{y_i \neq \hat{f}^b(x_i)\}}{\sum_{i=1}^n w_i}$$

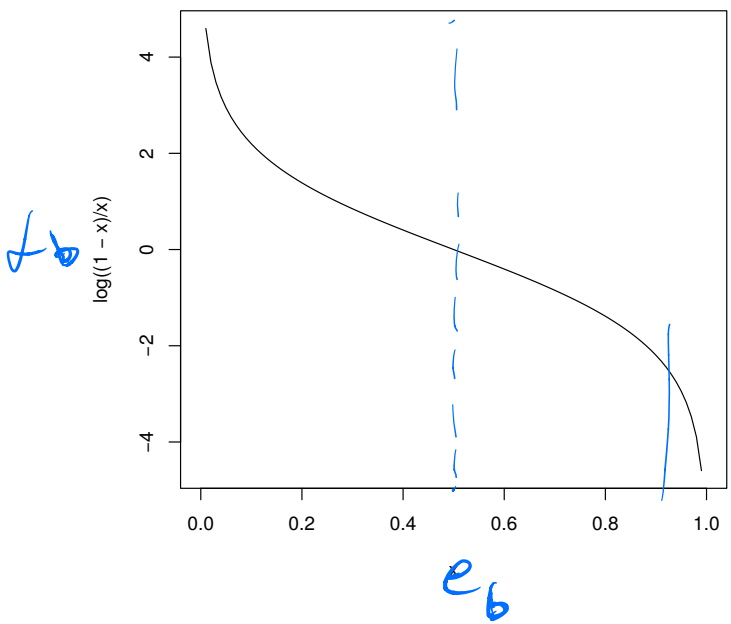
3. Define $\alpha_b = \log \frac{1-e_b}{e_b}$
4. Update the observation weights:

$$w_i \leftarrow w_i \cdot \exp\left(\alpha_b \mathbf{1}\{y_i \neq \hat{f}^b(x_i)\}\right)$$

Result: $\hat{f}(x) = \text{sign}\left(\sum_{b=1}^B \alpha_b \hat{f}^b(x)\right)$

AdaBoost: Step 3

$$\alpha_b = \log \frac{1-e_b}{e_b}.$$



This step stretches out probabilities near 0 and 1. It should remind you of logistic regression.

AdaBoost: Step 4

$$w_i \leftarrow w_i \cdot \exp\left(\alpha_b \mathbf{1}\{y_i \neq \hat{f}^b(x_i)\}\right)$$

If y_i was guessed wrong, multiply weight by e^{α_b}

If y_i was guessed right, multiply weight by 1

If α_b is big, the classifier is did its job well

If α_b is big, the weights increase lots.

Adaboost intuition

We have enough for an intuition. Adaboost builds a sequence of trees, each of which tries to classify well on points that were missed by the earlier trees.

The observation weights, w_i, focus later classifiers on difficult points.

The classifier weights α_b capture how well each component classifier does its weighted task, and is used for both

- Adjusting the observation weights
- Weighting the components in the final classifier

But why exactly these weights and functions??

Why this form for AdaBoost?

Suppose that we care about misclassification error, or 0-1 loss. When $Y \in \{-1, 1\}$, we can rewrite this.

$$\hat{f}(x) = \underset{f}{\operatorname{argmin}}\, \mathbb{E}\mathbf{1}\{Y \neq f(X)\} = \underset{f}{\operatorname{argmin}}\, \mathbb{E}\left(\mathbf{1}\{Yf(X) < 0\}\right)$$

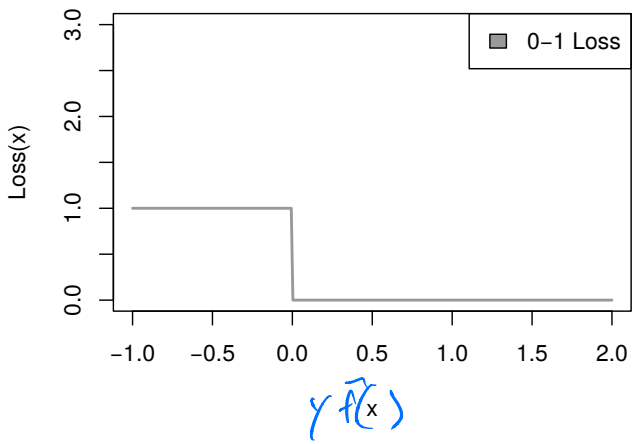
We've constrained ourselves to make $f(X)$ be an additive model built out of simple trees, so the sample version of this problem becomes

$$\hat{f}(x) = \underset{\alpha_b, f^{(b)}}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\left\{ y_i \left(\sum_{b=1}^{B} \alpha_b f^{(b)}(x_i) \right) < 0 \right\}$$

Now it just looks like an optimization problem! But how do we optimize it?

$\mathbb{1}(y\hat{f}(x) < 0)$ Approximating 0-1 loss

It turns out that 0-1 loss is not a very nice function to optimize



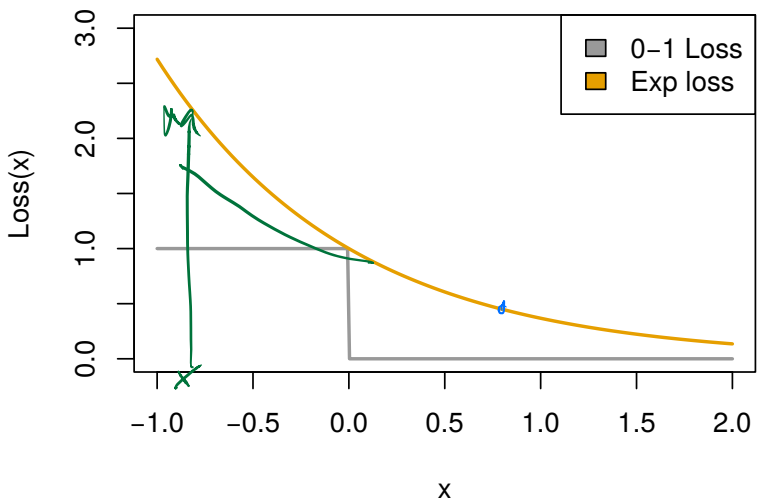
It's not differentiable or continuous. It's also constant except at 0!

Without being able to take a derivative, it's hard to tell what local changes to your function will improve the fit.

With regions of constant loss, there's not a sense of "close to correct," so it's hard to know which points are close to being correctly classified.

Approximating 0-1 loss

We switch 0-1 loss for another function that's easier to optimize and has similar behavior in important ways



This new function is continuous and differentiable and convex, and it is still monotonically decreasing.

If we can make the exponential function small, our 0-1 loss will also be good.

With the new loss

Making that switch, instead of solving

$$\underset{\{\alpha_b, f^{(b)}\}}{\operatorname{argmin}} \sum_{i=1}^{n} \mathbf{1}\left\{ y_i \left(\sum_{b=1}^{B} \alpha_b f^{(b)}(x_i) \right) < 0 \right\}$$

we just have to solve

$$\underset{\{\alpha_b, f^{(b)}\}}{\operatorname{argmin}} \sum_{i=1}^{n} \exp\left(-y_i \left(\sum_{b=1}^{B} \alpha_b f^{(b)}(x_i) \right) \right)$$

With the new loss

We just have to solve

$$\underset{\{\alpha_b, f^{(b)}\}}{\operatorname{argmin}} \sum_{i=1}^{n} \exp\left(-y_i \left(\sum_{b=1}^{B} \alpha_b f^{(b)}(x_i)\right)\right)$$

This is still tricky, so we cheat just a little more. We optimize one $\hat{f}^{(b)}$ at a time while holding the previous functions fixed, and never come back.

$$\underset{\alpha_b, \hat{f}^b}{\operatorname{argmin}} \sum_{i=1}^{n} \exp\left(-y_i \cdot \left(\sum_{k=1}^{b-1} \alpha_k \hat{f}^k(x) + \alpha_b \hat{f}^b(x)\right)\right)$$

This is a *greedy* algorithm. It seeks immediate rewards, rather than optimizing the overall objective.

$$\underset{\alpha_b, f^b}{\operatorname{argmin}} \sum_{i=1}^{n} \exp\left(-y_i \left(\underbrace{\sum_{k=1}^{b-1} \alpha_k \hat{f}^k(x)}_{\text{const.}} + \alpha_b \hat{f}^b(x)\right)\right)$$

$$\underset{\alpha_b, f^b}{\operatorname{argmin}} \sum_{i=1}^{n} \underbrace{\exp\left(-y_i \left(\sum_{k=1}^{b-1} \alpha_k \hat{f}^k(x)\right)\right)}_{\text{const.}} w_i^{(b)} \cdot \exp\left(-y_i \left(\alpha_b \hat{f}^b(x)\right)\right)$$

$$\underset{\alpha_b, f^b}{\operatorname{argmin}} \sum_{i=1}^{n} w_i^{(b)} \exp\left(-y_i \left(\alpha_b \hat{f}^b(x)\right)\right)$$

$$= \sum_{y_i = \hat{f}^b(x)} w_i^{(b)} e^{-\alpha_b} + \sum_{y_i \neq \hat{f}^{(b)}(x_b)} w_i^{(b)} e^{\alpha_b}$$

$$= \left(e^{\alpha_b} - e^{-\alpha_b}\right) \sum w_i^{(b)} I\{y_i \neq f^b(x)\} + e^{-\alpha_b} \sum_{i=1}^{n} w_i^{(b)}$$

$$\alpha = \log \frac{1 - e_b}{e_b}$$

$$\underset{\alpha_b}{\operatorname{argmin}} \left(e^{\alpha_b} - e^{-\alpha_b}\right) e_b - e^{-\alpha_b}$$

$$\left(e^{\alpha_b} + e^{-\alpha_b}\right) e_b - e^{-\alpha_b} = 0$$

$$+ e^{\alpha_b} e_b = e^{-\alpha_b}(1 - e_b)$$

$$+ e^{2\alpha_b} = \frac{1 - e_b}{e_b}$$

$$\alpha_b = \frac{1}{2} \log \frac{1 - e_b}{e_b}$$

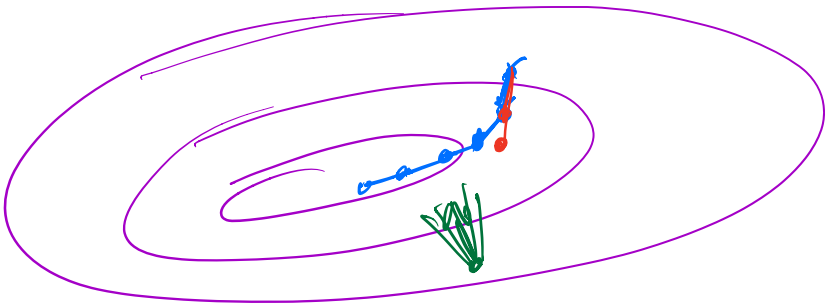
Oddly enough, optimizing this objective leads to the entire AdaBoost algorithm with all the weights and transformations!

[If we have time, show this by hand]

Improvements

This intuition automatically leads to many improvements

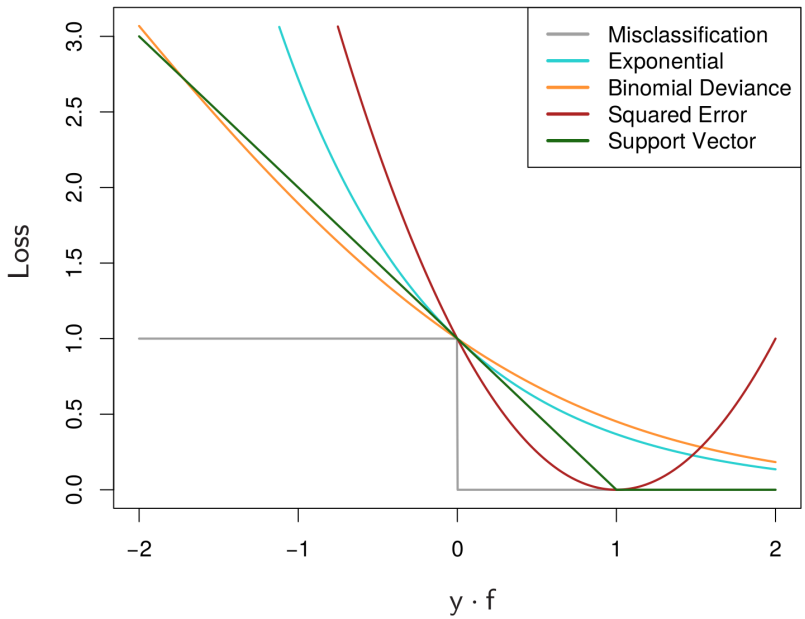
- *Shrinkage:* Only add a small amount of each tree at a time. Like taking smaller steps. Provides regularization. Use $\nu\alpha_b\hat{f}^b$ with $\nu \in [0, 1]$.
- *Subsampling:* Each step only sees a fraction, η of the data. Gives improvements to variance (and thus performance), as well as speed!
- *Other losses:* Why exponential loss?



$$\mathcal{L} = \sum_{i=1}^{n} \ell(x_i, \theta)$$

$$\frac{\partial \mathcal{L}}{\partial \theta_j} = \sum_{i=1}^{n} \frac{\partial \ell}{\partial \theta_j}(x_i, \theta)$$

Improvements: Why exponential loss?



(ESL Figure 10.4)

There are many losses that approximate 0-1 loss. Some can give dramatically better performance in certain settings.

Improvements: Why exponential loss?

It turns out that the exponential loss leads to a particularly simple algorithm, while the others do not. The optimization becomes harder, and the updates lose their beautiful form.

Instead, at iteration b, the gradient $g_b \in \mathbb{R}^n$ of the loss function is found. The new classifier $\hat{f}^{(b)}$ is then fit to approximate this gradient vector well. Hence the name: *Gradient Boosting*.

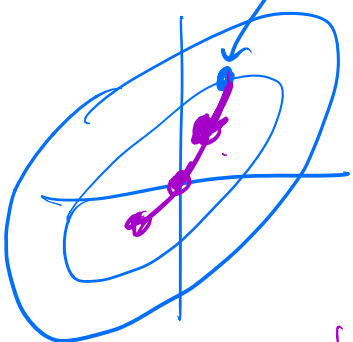
Right now, I've found one of the best implementations to be `xgboost`, which luckly exists in both R and python (and other languages).

Gradient boosting

$b \in \mathbb{R}^n$

$$L \quad \sum_{i=1}^{n} \underline{\text{Loss}}\left(y_i, \sum_{k=1}^{b-1} \alpha_k f^k(x) + b_i\right)$$

q



$$\nabla L \in \mathbb{R}^n = \left(\frac{\partial L}{\partial b_1}, \frac{\partial L}{\partial b_2}, \ldots\right)$$

$$\underbrace{\sum_{k=1}^{b-1} \alpha_k f^k(x)}_{p} + \underbrace{\sum_{i=1}^{n} \mathbb{1}\{x = x_i\} \frac{\partial L}{\partial b_i}(\varepsilon)}_{q}$$

(Don't do this)

$$f^b(x_i) \approx (\nabla_y L)_i$$

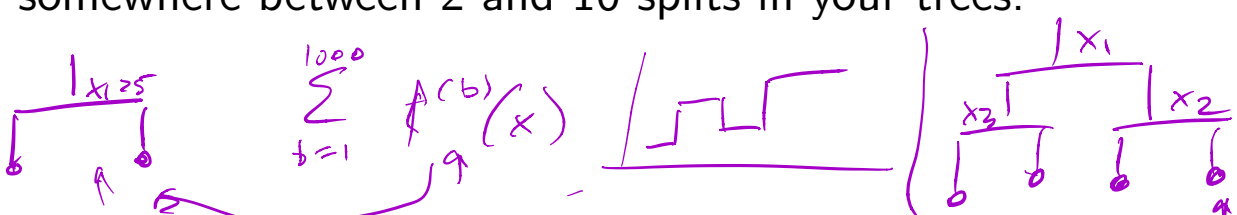
Boosting discussion

In bagging/random forests, we use deep trees to stay unbiased, and then rely on averaging to reduce variance.

In boosting, we tend to use very simple estimators, like very shallow trees. These have low variance, but are high bias. Because the sequence of trees can adjust for previous errors, they can fix the bias!

The shallower trees can also lead to computational speedups in evaluation, since you don't have to evaluate 500 deep trees.

Think of the depth of your trees as allowing interactions. Two splits let you interact two variables. You'll often find that you want somewhere between 2 and 10 splits in your trees.



Boosting discussion

Boosting is one of the most powerful classical methods. When well-tuned, it can beat random forests and most other classifiers.

However, it requires more tuning than random forests.

Tuning parameters:

- Number of trees, B
- Amount of subsampling, η
- Amount of shrinkage, ν
- Number of splits in each tree

You will find that random forest are difficult to badly overfit. In boosting, choosing B too large can overfit. Choosing it too small can give a bad classifier.

B is the main tuning parameter. The others can be tuned more roughly, since it doesn't matter quite as much.

Empirical risk minimization

This general pattern:

1. We want to minimize:

$$\mathbb{E}\mathbf{1}\{Y \neq f(X)\}$$

2. And so we actually try to minimize its empirical version:

$$\frac{1}{n}\sum_{i=1}^{n}\mathbf{1}\{y_i \neq f(x_i)\}$$

3. But we can't even do that for classification, because it is NP-hard. So we introduce a nicer loss L and minimize

$$\frac{1}{n}\sum_{i=1}^{n}L(y_i, f(x_i))$$

The first two steps are known as *empirical risk minimization*. The last step almost always follows for classification.

Adaboost final summary

Adaboost is just

1. Empirical risk minimization with an exponential loss

$$\frac{1}{n}\sum_{i=1}^{n} e^{-y_i f(x_i)}$$

2. Assuming an additive model of trees:

$$\hat{f}(x) = \sum_{b=1}^{B} \alpha_b \hat{f}^{(b)}(x)$$

   with $\hat{f}^{(b)}(x)$ constrained to be a tree.

3. And greedy, stepwise optimization

$$\underset{\alpha_b, \hat{f}^b}{\operatorname{argmin}} \sum_{i=1}^{n} \operatorname{Exp}\left(-y_i \cdot \left(\sum_{k=1}^{b-1} \alpha_k \hat{f}^k(x) + \alpha_b \hat{f}^b(x)\right)\right)$$

Empirical risk minimization

Different choices of L lead to different methods and different trade-offs.

Exponential loss:

$$L(y_i, f(x_i)) = e^{-y_i f(x_i)}$$

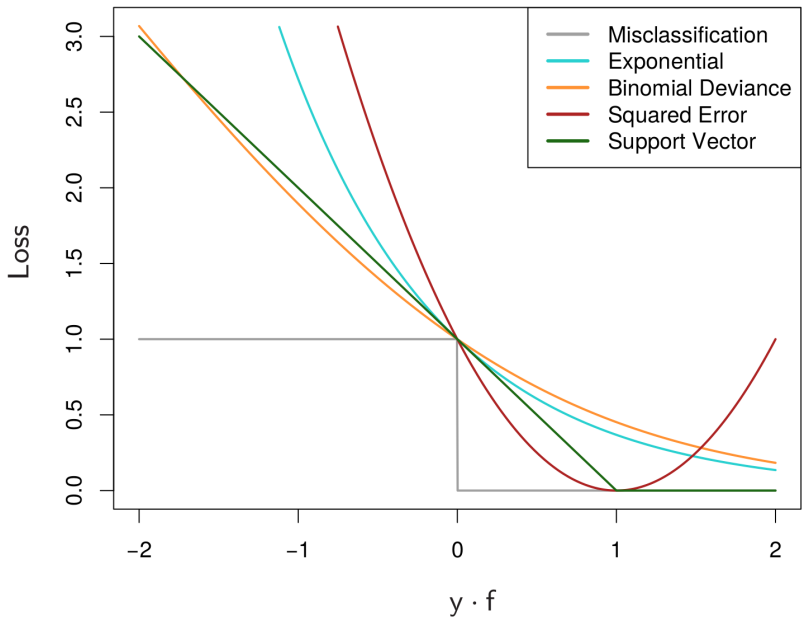
Logistic loss:

$$L(y_i, f(x_i)) = \log\left(1 + e^{-y_i f(x_i)}\right)$$

Hinge loss (SVM):

$$L(y_i, f(x_i)) = \max\left\{0, 1 - y_i f(x_i)\right\}$$

Common losses



(ESL Figure 10.4)

Ensemble Learning

One last comment on combining classifiers.

In boosting and bagging, we saw two approaches to combining many weaker predictors into a much stronger predictor

This combination is called an *ensemble*. There are many general approaches to building ensembles.

Ensembles can combine methods of different types with different strengths, so that each can perform well in cases where it is strong.

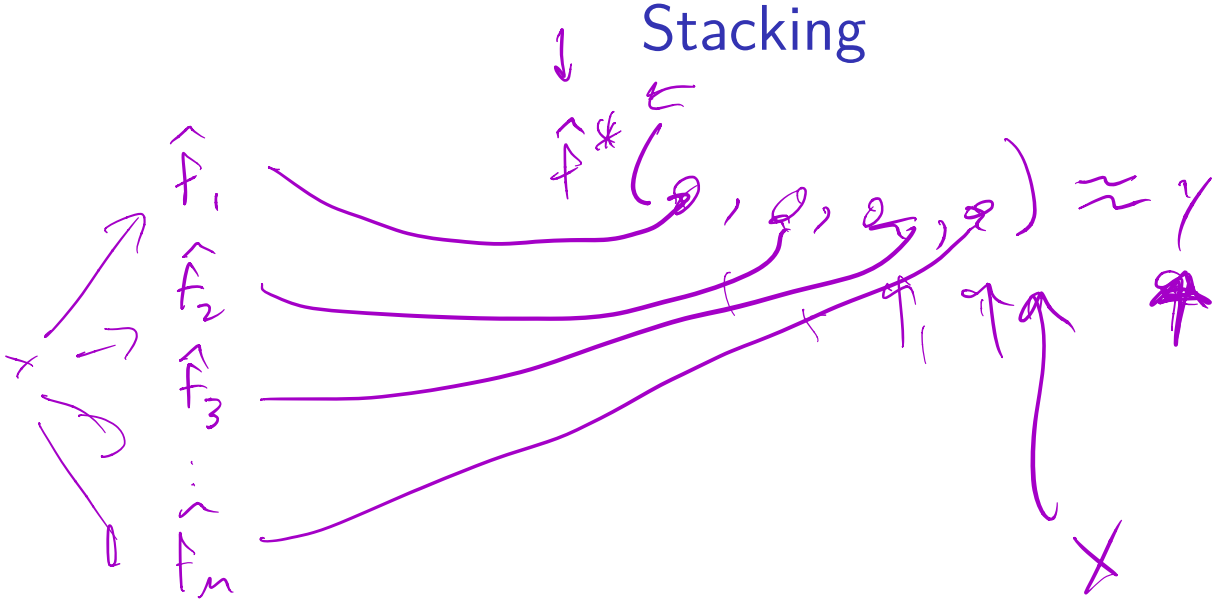
Many contests are won in this way. In practice, there are sometimes practical concerns about complexity and speed.

How can we combine classifiers?

Suppose that I build M different classifiers on my data, $\hat{f}_1, \ldots, \hat{f}_M$. How can I combine their guesses?

- Voting, weighted voting
- Averaging
- Stacking:

Stacking



$$\hat{Y} = \sum_{k=1}^{m} \beta_k \hat{f}_k(x)$$